

Spring Boot Core Concepts

IoC, Dependency Injection,
Beans, and the Spring Container



Inversion of Control

an application is designed to receive its dependencies from an external source rather than creating them

this contrasts with traditional software design, where an application manages its own dependencies





inversion of control is implemented using dependency injection with the help of a spring container

instead of directly creating dependencies within a class, they are provided from an external source: the Spring Container



IoC vs DI

Inversion of Control

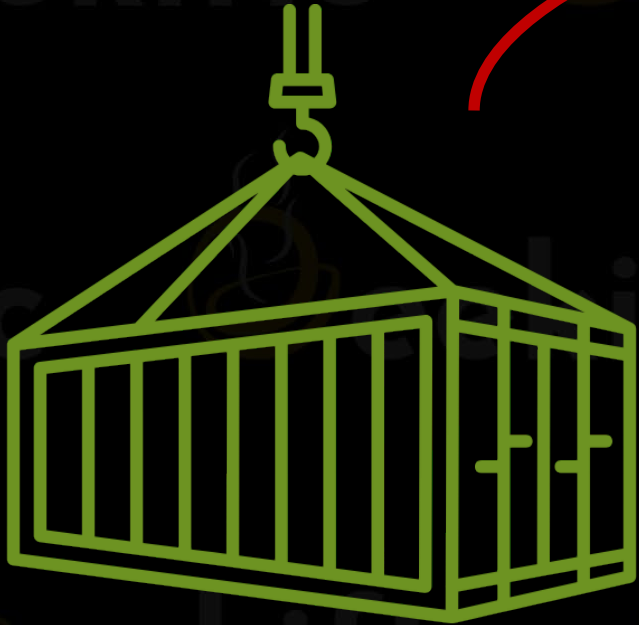
a design principle in which a software component is designed to receive its dependencies from an external entity

Dependency Injection

a programming technique that focuses on managing the dependencies required by these components, which are components themselves



Dependency Injection



Spring Container
(ApplicationContext)

```
public VideoService(IRepository repository) {  
    this.repository = repository;  
}
```

**instantiates, assembles, manages,
and stores the components of our
application**



a component

```
@Service
@RequiredArgsConstructor
public class VideoService implements IVideoService {

    @Qualifier("jpa")
    private final IRepository repository;

    ...
}
```

its dependency
(a component itself, with
its own dependencies)



managed by the
spring container

→ Bean

```
@Service
@RequiredArgsConstructor
public class VideoService implements IVideoService {

    @Qualifier("jpa")
    private final IRepository repository;

    ...

}
```



Beans

the backbone of a Spring application
plain old Java objects (POJOs) that can be enhanced with AOP

Spring Container manages these beans and provides
a runtime environment to interact with each other



BeanFactory & ApplicationContext

BeanFactory

central interface for accessing and managing beans, it provides basic bean instantiation and dependency resolution capabilities

ApplicationContext

extends the functionality of the BeanFactory offering additional features such as the integration with other Spring modules



Beans

declared and configured using various mechanisms including XML-based, Java-based and annotation-based configuration

the most common approach is the annotation-based one where beans are annotated with Spring Stereotypes such as `@Component`, `@Service`, `@Repository`, or `@Controller`

wired together using dependency injection, where the dependency bean is injected into the dependent bean



Injection Types

Constructor Injection is the recommended approach – dependencies are passed through the constructor, making them required and immutable

Field Injection uses `@Autowired` directly on fields – convenient but harder to test. Setter Injection offers optional dependencies via setter methods

Spring recommends constructor injection for mandatory dependencies and setter injection for optional ones



Stereotypes in Practice

each Spring stereotype signals a specific role –
`@Controller` handles HTTP requests, `@Service` contains
business logic, `@Repository` manages data access

`@Component` is the generic stereotype – the others
are specializations that enable additional framework
behavior like exception translation

the following code examples show how these annotations
mark each class and how dependencies flow between layers



@RestController

@RequiredArgsConstructor

```
public class VideoController implements VideoApi {  
  
    private final IVideoService service;  
    private final IVideoApiMapper mapper;  
  
    ...  
}
```

each class is marked with a stereotype annotation
– @RestController, @Service, @Repository,
@Component – that tells Spring its role

@Service

@RequiredArgsConstructor

```
public class VideoService implements IVideoService {  
  
    @Qualifier("jpa")  
    private final IRepository repository;  
  
    ...  
}
```

@Repository("jpa")

@RequiredArgsConstructor

```
public class PostgresVideoRepository implements IRepository {  
  
    private final JpaVideoRepository jpaVideoRepository;  
    private final IVideoJpaMapper mapper;  
  
    ...  
}
```

@Component

```
public class VideoJpaMapper implements IVideoJpaMapper {  
  
    ...  
}
```



`@RequiredArgsConstructor` generates a constructor for all final fields – Spring uses this for constructor-based injection

```
@RestController
@RequiredArgsConstructor
public class VideoController implements VideoApi {

    private final IVideoService service;
    private final IVideoApiMapper mapper;

    ...

}
```

```
@Service
@RequiredArgsConstructor
public class VideoService implements IVideoService {

    @Qualifier("jpa")
    private final IRepository repository;

    ...

}
```

```
@Repository("jpa")
@RequiredArgsConstructor
public class PostgresVideoRepository implements IRepository {

    private final JpaVideoRepository jpaVideoRepository;
    private final IVideoJpaMapper mapper;

    ...

}
```

```
@Component
public class VideoJpaMapper implements IVideoJpaMapper {

    ...

}
```



```
@RestController
public class VideoController implements VideoApi {

    @Autowired
    private final IVideoService service;

    @Autowired
    private final IVideoApiMapper mapper;

    ...
}
```

@Autowired injects dependencies directly into fields – an alternative to constructor injection, though less recommended

```
@Service
@RequiredArgsConstructor
public class VideoService implements IVideoService {

    @Qualifier("jpa")
    private final IRepository repository;

    ...
}
```

```
@Repository("jpa")
@RequiredArgsConstructor
public class PostgresVideoRepository implements IRepository {

    private final JpaVideoRepository jpaVideoRepository;
    private final IVideoJpaMapper mapper;

    ...
}
```

```
@Component
public class VideoJpaMapper implements IVideoJpaMapper {

    ...
}
```



```
@RestController
public class VideoController implements VideoApi {

    @Autowired
    private final IVideoService service;

    @Autowired
    private final IVideoApiMapper mapper;

    ...

}
```

@Qualifier disambiguates when multiple beans implement the same interface – here "jpa" selects the PostgresVideoRepository

```
@Service
@RequiredArgsConstructor
public class VideoService implements IVideoService {

    @Qualifier("jpa")
    private final IRepository repository;

    ...

}
```

```
@Repository("jpa")
@RequiredArgsConstructor
public class PostgresVideoRepository implements IRepository {

    private final JpaVideoRepository jpaVideoRepository;
    private final IVideoJpaMapper mapper;

    ...

}
```

```
@Component
public class VideoJpaMapper implements IVideoJpaMapper {

    ...

}
```



Application Startup

when a Spring Boot application starts, the framework scans for annotated classes and registers them as beans in the `ApplicationContext`

it resolves all dependencies, wires beans together via injection, and initializes the entire object graph before the app is ready to serve requests

the console output confirms each bean being created, injected, and the embedded server starting up



Spring Boot console – scanning classes, wiring beans, and launching the embedded server

```

 ____  _
 / __ \| | | |
( (___| |_| |
 \___ \|  _/ |
      | | | |
      |_| |_|
=====|_|=====|_|/=//_/_/_/
:: Spring Boot ::      (v2.7.4)

```

```

INFO 12388 --- [main] com.youtube.geekific.Application : Starting Application using Java 17.0.1 with PID 12388
INFO 12388 --- [main] com.youtube.geekific.Application : No active profile set, falling back to 1 default profile
INFO 12388 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
INFO 12388 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 57 ms
INFO 12388 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
INFO 12388 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
INFO 12388 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.65]
INFO 12388 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
INFO 12388 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
INFO 12388 --- [main] o.f.c.internal.license.VersionPrinter : Flyway Community Edition 9.8.1 by Redgate
INFO 12388 --- [main] o.f.c.internal.license.VersionPrinter : See what's new here: https://flywaydb.org/documentation/9.8.1
INFO 12388 --- [main] o.f.c.internal.license.VersionPrinter :
INFO 12388 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
INFO 12388 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
INFO 12388 --- [main] o.f.c.i.database.base.BaseDatabaseType : Database: jdbc:postgresql://localhost:5432/postgres (PostgreSQL 15.3)
INFO 12388 --- [main] o.f.core.internal.command.DbValidate : Successfully validated 1 migration (execution time 00:00.048s)
INFO 12388 --- [main] o.f.core.internal.command.DbMigrate : Current version of schema "public": 1.1
INFO 12388 --- [main] o.f.core.internal.command.DbMigrate : Schema "public" is up to date. No migration necessary.
INFO 12388 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
INFO 12388 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 5.6.11.Final
INFO 12388 --- [main] o.hibernate.annotations.common.Version : HCANN000001: Hibernate Commons Annotations {5.1.2.Final}
INFO 12388 --- [main] org.hibernate.dialect.Dialect : HHH000400: Using dialect: org.hibernate.dialect.PostgreSQLDialect
INFO 12388 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
INFO 12388 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
WARN 12388 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly co
INFO 12388 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint(s) beneath base path '/actuator'
INFO 12388 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
INFO 12388 --- [main] com.youtube.geekific.Application : Started Application in 6.314 seconds (JVM running for 6.791)

```



Inversion of Control is the principle of relinquishing control of our application's components to an external entity

Dependency Injection is the programming technique that helps us manage the dependencies between these components

IoC and DI enable developers to build modular, maintainable, and scalable applications

in SpringBoot's case, Beans and the Spring Container embodies these principles

